

Local AI Memory Assistant

Phases, Terms and Importance of Each Implementation

Technical documentation focused on reuse: what each component does, why it exists, what it measures, how it integrates, and when it is valuable in other AI, search, memory and recommendation systems.

Scope. This document describes the project by functional phases: retrieval evaluation, hybrid search, LTR ranking, reranking, quality scoring, duplicate detection, diversity, feedback, online/offline learning, model persistence, context, knowledge graph, lifecycle, jobs, observability and testing.

Core principle: the value of the project is not a single technology. It is the complete pipeline that turns persisted memory into candidates, ranks them with multiple signals, learns from feedback and measures whether quality actually improves.

1. Global pipeline overview

The canonical retrieval path can be summarized as: Query → analysis/strategy → Hybrid Retrieval → fusion → LTR Ranking → Reranking → Quality Scoring → Duplicate Detection → Diversity Filtering → Feedback Reranking → metrics → feedback → training → model persistence.

Phase	Input	Main output	Goal
1. Query analysis	Query	Profile / strategy	Choose retrieval behavior
2. Hybrid Retrieval	Query + strategy	Candidates	Maximize coverage
3. Fusion / RRF	Partial rankings	Fused ranking	Combine sources
4. LTR Ranking	Candidates + features	Learned score	Optimize ordering
5. Reranking	Top candidates	Refined order	Improve final relevance
6. Quality Scoring	Candidate	Quality score	Reject weak context
7. Duplicate Detection	Results	Unique results	Remove redundancy
8. Diversity Filtering	Results	Diverse list	Increase information coverage
9. Feedback Reranking	Historical feedback	Adjusted ranking	Learn preferences
10. Evaluation	Ranking + ground truth	P@K/R@K/MRR/NDCG	Measure quality

2. Ranking evaluation metrics

These metrics are critical because they turn subjective impressions into measurable quality. Without them, changing weights, adding LTR or introducing context may look better while actually degrading retrieval.

Precision@K

What it is: The proportion of relevant results among the first K results. $\text{Precision@K} = \frac{\text{relevant_in_top_K}}{K}$

Importance in this project: Measures how clean the visible portion of the ranking is.

Value in other projects: Useful in search, RAG, recommendation and memory systems where users inspect only a small top-K.

Recall@K

What it is: The proportion of all available relevant results retrieved within top K. $\text{Recall@K} = \frac{\text{relevant_in_top_K}}{\text{total_relevant}}$

Importance in this project: Measures coverage and exposes cases where important memories are missed.

Value in other projects: Critical when missing relevant information is costly.

MRR — Mean Reciprocal Rank

What it is: Average reciprocal rank of the first relevant result. Rank 1 contributes 1.0; rank 2 contributes 0.5.

Importance in this project: Measures how quickly the first useful answer appears.

Value in other projects: Ideal for assistants, QA, entity search and first-result-sensitive systems.

NDCG@K

What it is: Normalized Discounted Cumulative Gain rewards graded relevance and discounts relevant items that appear too late.

Importance in this project: Captures the quality of the complete ordering better than binary precision alone.

Value in other projects: Especially useful for LTR, recommendation and enterprise search.

Combined interpretation: Precision@K measures purity, Recall@K coverage, MRR first-hit position, and NDCG@K overall ordering quality.

3. Hybrid Retrieval

Vector / Semantic Retrieval

What it is: Embedding-based retrieval using vector similarity.

Importance in this project: Finds conceptually related memories even when exact words differ.

Value in other projects: Foundation for RAG, semantic search and assistant memory.

Keyword / BM25 Retrieval

What it is: Lexical retrieval favoring exact or statistically important term matches.

Importance in this project: Protects names, identifiers, acronyms and exact phrases that embeddings can underweight.

Value in other projects: Important for technical docs, code, logs and catalogs.

Graph Retrieval

What it is: Retrieval from knowledge-graph nodes and relationships.

Importance in this project: Adds relational evidence beyond textual similarity.

Value in other projects: Useful for dependencies, entities, multi-hop reasoning and relational recommendation.

Graph Evidence Retrieval

What it is: A feature representing support derived from graph relations or inferred knowledge.

Importance in this project: Allows structurally supported candidates to rank higher even with weaker text similarity.

Value in other projects: Valuable where relationships are first-class evidence.

Weighted Reciprocal Rank Fusion (Weighted RRF)

What it is: Combines rankings using rank positions plus source-specific weights.

Importance in this project: Robustly merges vector, keyword, graph and graph-evidence results without requiring scores to share a scale.

Value in other projects: Reusable whenever heterogeneous retrieval engines must be fused.

4. Adaptive retrieval strategy

Query Profile

What it is: Structured query characteristics such as token count, specificity, complexity, semantic/keyword/relational/temporal intent, entities and comparison intent.

Importance in this project: Allows retrieval behavior to change according to query type.

Value in other projects: Reusable in any multi-index search engine.

Retrieval Strategy

What it is: Derived configuration containing mode, vector/keyword/graph weights, topK, expansion, rerank and temporal boost.

Importance in this project: Controls candidate budget and the precision/recall tradeoff before expensive ranking.

Value in other projects: Useful when exact, semantic, relational and temporal queries need different treatment.

Candidate Budget

What it is: Number of candidates retrieved before final ranking.

Importance in this project: Ambiguous queries can receive broader coverage while exact queries remain efficient.

Value in other projects: Controls latency/cost while preserving recall.

5. Learning to Rank (LTR)

FeatureExtractor

What it is: Transforms every query-candidate pair into an explicit numeric feature vector.

Importance in this project: Separates signal generation from the model and makes ranking inspectable.

Value in other projects: Allows models to evolve without rewriting retrieval.

Feature	Meaning	Ranking contribution
semantic	Query-memory semantic similarity	Conceptual relevance
bm25	Lexical match score	Exact term relevance
importance	Accumulated memory importance	Knowledge priority
confidence	Memory confidence	Downweight uncertain data
freshness	Recency	Prefer current information
graphEvidence	Graph support	Relational evidence
accessCount	Historical use	Recurring utility
diversity	Diversity contribution	Reduce thematic redundancy
duplicatePenalty	Duplicate penalty	Suppress repeated candidates
feedbackScore	Aggregated feedback signal	Behavioral learning
retrievalFrequency	Retrieval frequency	Usage signal
ageScore	Transformed memory age	Temporal decay control

LinearModel / LTRRanker

What it is: A model combining features with learned weights to produce the final ranking score.

Importance in this project: Moves ranking beyond fully manual heuristics while remaining interpretable.

Value in other projects: A strong starting point before more complex rankers.

PersistentLTRModelProvider + ModelRepository

What it is: Loads the persisted model and provides it to the serving ranker.

Importance in this project: Ensures the last trained model survives restarts instead of reverting to defaults.

Value in other projects: Reusable in any ML serving architecture requiring durable learned state.

Example training feature vector

query	angular federation
memoryId	memory-001
semantic	0.92
bm25	0.81
importance	0.70
confidence	0.90
freshness	0.95
graphEvidence	0.61
accessCount	0.42
diversity	0.84
duplicatePenalty	0.05
label	1

The label is the learning target. The project also uses reward signals such as click, accept, reject and ignore, so feedback semantics must distinguish exposure from explicit user action.

6. Reranking

Embedding Reranker

What it is: Reorders a small candidate set using a more precise or expensive semantic signal.

Importance in this project: Keeps expensive computation limited to top candidates.

Value in other projects: Standard two-stage retrieval pattern: retrieve cheaply, rerank accurately.

Semantic Reranker

What it is: Refines ranking according to semantic relevance after fusion.

Importance in this project: Corrects cases where fusion preserves coverage but not the ideal final order.

Value in other projects: Reusable in RAG, search and recommendation.

FeedbackDrivenReranker

What it is: Adjusts results using historical feedback.

Importance in this project: Lets ranking reflect actual usage rather than only static model assumptions.

Value in other projects: Useful for personalization and adaptive search.

7. Quality Scoring

QualityScoringService

What it is: Produces a quality assessment separate from pure query relevance.

Importance in this project: A candidate can be relevant but unreliable, stale or otherwise poor context.

Value in other projects: Useful in RAG to prevent weak evidence from reaching the LLM.

8. Duplicate Detection and Diversity Filtering

DuplicateDetector

What it is: Detects identical or highly similar results using text/semantic similarity.

Importance in this project: Prevents duplicate memories from consuming scarce top-K positions.

Value in other projects: Critical in cumulative memory and repeated-ingestion systems.

Duplicate Penalty

What it is: Feature/penalty applied to candidates that resemble other results.

Importance in this project: Allows LTR to learn to demote redundant items before hard filtering.

Value in other projects: Balances useful variants against true duplicates.

DiversityService / Diversity Filtering

What it is: Selects a final set with lower semantic or thematic redundancy.

Importance in this project: Increases information coverage inside a limited top-K.

Value in other projects: Valuable for RAG, recommendations and exploratory search.

Difference: deduplication removes near-equivalents; diversity prevents distinct results from all saying essentially the same thing.

9. Feedback and learning

FeedbackCollector

What it is: Captures query, memory, feedback type, features and learning signal.

Importance in this project: Connects user behavior to ranking improvement.

Value in other projects: The bridge from static ranking to adaptive ranking.

FeedbackRepository

What it is: Persists feedback events.

Importance in this project: Enables offline training, auditing and dataset reconstruction.

Value in other projects: Core pattern in search and recommendation systems.

FeedbackService

What it is: Business layer for recording and reading feedback.

Importance in this project: Decouples endpoints and pipelines from storage details.

Value in other projects: Makes persistence and validation replaceable.

FeedbackLearningService

What it is: Orchestrates feedback and online model updates.

Importance in this project: Turns user signals into controlled learning updates.

Value in other projects: Reusable when a product must adapt continuously.

Reward Model

What it is: Maps actions to numeric reward; project tests include examples such as click=0.30, accept=1.00, reject=-1.00 and ignore=-0.20.

Importance in this project: Gives heterogeneous interactions a common learning meaning.

Value in other projects: Useful for online learning, bandits and adaptive ranking.

Critical semantic risk: an impression must not automatically be recorded as a click or acceptance. Otherwise the system learns false positive feedback and can reinforce its own ranking bias.

10. Offline and online training

TrainingService — Offline Training

What it is: Retrains/recalculates the model from accumulated feedback.

Importance in this project: Supports reproducible experiments, larger datasets and controlled releases.

Value in other projects: Suitable for batch retraining and model governance.

OnlineTrainer

What it is: Applies incremental updates as signals arrive.

Importance in this project: Shortens the delay between feedback and adaptation.

Value in other projects: Useful for dynamic preferences and fast-changing domains.

OnlineOptimizer

What it is: Computes weight updates from error/reward signals.

Importance in this project: Encapsulates optimization math and makes the optimizer replaceable.

Value in other projects: Reusable for SGD, gradient methods, bandits or custom optimizers.

Learning Rate

What it is: Controls the magnitude of each online update.

Importance in this project: Prevents a single interaction from changing the ranking too aggressively.

Value in other projects: Essential for balancing adaptation and stability.

11. Model persistence and restart recovery

Component / capability	Status
FeedbackCollector	Complete
FeedbackRepository	Complete
FeedbackService	Complete
FeedbackLearningService	Complete
OnlineTrainer	Complete
OnlineOptimizer	Complete
ModelRepository	Complete
Model persistence	Complete
Restore after restart	Complete
LTRRanker integration	Complete
Old loader removal	Complete
Regression tests	Complete

Model persistence is critical because it separates training from serving. A trained model that disappears after restart is not operational learning.

Production persistence must also cover feedback, knowledge graph, inference state and other operational data, not only LTR weights.

12. Ranking regression tests

Regression Ranking Tests

What it is: Versioned query datasets with expected relevance and minimum metric thresholds.

Importance in this project: Prevent a local feature improvement from silently degrading previously correct queries.

Value in other projects: Essential when embeddings, weights, rerankers, context or indexes change.

The recommended gate compares baseline vs candidate using Precision@K, Recall@K, MRR, NDCG@K and latency.

13. Context-aware retrieval

ContextModel

What it is: Structured session/query context containing entities, topics, goals, constraints and temporal signals.

Importance in this project: Provides richer retrieval information than the literal query text.

Value in other projects: Highly valuable in multi-turn assistants, agents and copilots.

ContextBuilder / Context Orchestrator

What it is: Builds the context consumed by retrieval and generation.

Importance in this project: Separates context extraction from context usage.

Value in other projects: Reusable in conversational RAG and stateful systems.

A key integration requirement is ensuring the constructed ContextModel reaches the real HTTP retrieval path without changing behavior when context is absent.

14. Knowledge Graph and inference

KnowledgeRepository

What it is: Persists extracted or consolidated knowledge.

Importance in this project: Separates raw memory from structured knowledge.

Value in other projects: Reusable when facts need normalization, reasoning or provenance.

KnowledgeSyncService

What it is: Synchronizes persisted knowledge with graph nodes and edges.

Importance in this project: Prevents graph retrieval and inference from operating on stale representations.

Value in other projects: Useful whenever two storage representations must remain coherent.

GraphRepository / GraphStorage

What it is: Provides graph access and persistence.

Importance in this project: Centralizes entity identity, relationships and storage.

Value in other projects: Foundation for local or enterprise knowledge graphs.

InferenceRepository / Multi-hop Inference

What it is: Stores or manages knowledge derived from relationships.

Importance in this project: Allows retrieval of inferred conclusions rather than only explicit facts.

Value in other projects: Useful in troubleshooting, dependency analysis and reasoning.

A critical project phase is the automatic flow knowledge extraction → persistence → graph synchronization → validation → inference.

15. Memory lifecycle and consolidation

Memory Lifecycle

What it is: Policies for importance, access count, last access, decay, archive and other memory-state transitions.

Importance in this project: Prevents all memories from remaining equally important forever.

Value in other projects: Reusable in persistent assistants and long-term knowledge systems.

Importance

What it is: Priority signal that can increase through use or reinforcement.

Importance in this project: Adds utility beyond query similarity.

Value in other projects: Useful for ranking, retention and consolidation.

Freshness / Age Score

What it is: Transforms memory age into a ranking feature.

Importance in this project: Allows recent information to be preferred without deleting historical knowledge.

Value in other projects: Useful in temporal and operational domains.

ConsolidationService

What it is: Merges or summarizes related memories.

Importance in this project: Controls fragmentation, redundancy and storage growth.

Value in other projects: Important in long-running assistants.

Cleanup Job

What it is: Removes or archives eligible memories under explicit policy.

Importance in this project: Controls unbounded growth but must be safe, observable and idempotent.

Value in other projects: Reusable in retention-sensitive systems.

Cleanup should begin with dry-run behavior, metrics and strict eligibility rules before destructive deletion is enabled.

16. Jobs, scheduler and operation

Scheduler

What it is: Plans periodic inference, relearning, cleanup or training tasks.

Importance in this project: Turns isolated capabilities into continuous operational behavior.

Value in other projects: Reusable for maintenance and ML pipelines.

runJob + JobRepository

What it is: Executes jobs while recording status/history.

Importance in this project: Improves observability and recovery.

Value in other projects: General pattern for background jobs and ETL.

Training Job

What it is: Scheduled execution of LTR training.

Importance in this project: Incorporates accumulated feedback without manual intervention.

Value in other projects: Reusable for periodic model recalibration.

Production maturity also requires retry/backoff, locking against overlapping runs, dependency ordering, persistent history and alerts.

17. Observability and operational metrics

MetricsService

What it is: Records durations, quality and pipeline outcomes.

Importance in this project: Shows where time is spent and which stage improves or degrades quality.

Value in other projects: Essential for optimizing retrieval and detecting regressions.

The project instruments stages such as Hybrid Retrieval, LTR Ranking, Reranking, Quality Scoring, Duplicate Detection and Diversity Filtering separately, making bottlenecks visible.

DashboardService

What it is: Aggregates operational metrics for inspection.

Importance in this project: Turns low-level telemetry into a usable health view.

Value in other projects: Reusable in ML/search services and job systems.

18. Persistence and production data

Qdrant / MemoryRepository

What it is: Vector persistence and embedding retrieval for memories.

Importance in this project: Supports semantic search at scale while separating storage from ranking logic.

Value in other projects: Reusable in RAG, semantic search and memory services.

KeywordIndex / KeywordIndexLoader

What it is: In-memory lexical index plus loader rebuilt from durable memory.

Importance in this project: Supports fast BM25/keyword retrieval while remaining synchronized with stored memories.

Value in other projects: Useful when a derived index is rebuilt from a durable source.

Operational data directory

What it is: Unified durable location for models, feedback, knowledge, graph, inference and state.

Importance in this project: Prevents data loss and environment inconsistencies.

Value in other projects: Essential when local artifacts coexist with external databases.

Container persistence is part of the architecture, not merely deployment plumbing. If model/feedback/graph files are not mounted durably, restarts can erase learned state.

19. API contracts and security

Canonical Express API

What it is: Active HTTP path for memory, context, knowledge, jobs, metrics and dashboard.

Importance in this project: Defines the service's external contract.

Value in other projects: Enables integration with UIs, agents, CLI and other services.

Payload Validation

What it is: Input schemas and normalized errors.

Importance in this project: Prevents invalid state and reduces attack surface.

Value in other projects: Required in any production API.

Authentication / Authorization

What it is: Identity and permission control for mutable endpoints.

Importance in this project: Protects memory, feedback, training and maintenance operations.

Value in other projects: Critical for personal or enterprise data.

20. Overall implementation / stabilization state

Area	State	Why it matters
Hybrid retrieval	Implemented	Multi-source coverage
LTR	Implemented	Learned, interpretable ranking
Reranking	Implemented	Final precision refinement
Quality / duplicate / diversity	Implemented	Top-K quality
Feedback + online learning	Implemented	Continuous adaptation
LTR persistence	Implemented	Durable learning
Ranking metrics	Implemented in tests	Objective validation
Context-aware retrieval	Partially integrated	Session-aware retrieval
Knowledge → graph sync	Integration pending	Graph coherence
Lifecycle / consolidation	Partial	Long-term memory health
Cleanup	Incomplete	Growth control
Reproducible suite + CI	Pending	Quality gate
Production persistence	Needs stabilization	Restart durability

21. Reuse value in other projects

Implementation	Problem solved	When to reuse
Precision/Recall/MRR/NDCG	Objective ranking evaluation	Any retrieval/recommender/RAG
Hybrid Retrieval	Semantic + exact + relational coverage	Heterogeneous corpora
Weighted RRF	Fuse heterogeneous rankings	Multiple engines/indexes
LTR	Learn result ordering	Feedback or ground truth exists
FeatureExtractor	Separate signals from model	Interpretable evolving ranking
Reranking	Improve top-N efficiently	Two-stage search
Quality Scoring	Reject weak candidates	Reliable RAG
Duplicate Detection	Remove redundancy	Repeated/cumulative ingestion
Diversity Filtering	Increase top-K coverage	Exploration/recommendation/RAG
FeedbackCollector	Capture real signals	Interactive products
OnlineTrainer	Adapt quickly	Changing preferences
Offline Training	Reproducible retraining	Release/model governance
ModelRepository	Durability/versioning	ML serving
ContextModel	Multi-turn personalization	Assistants/agents
Knowledge Graph	Relations and inference	Relational domains
Lifecycle/Consolidation	Sustainable long-term memory	Persistent agents
Scheduler/Jobs	Automated maintenance	Operational services
Regression Tests	Prevent silent degradation	Frequent ranking changes

22. Recommended phase order when replicating the system

Phase 1 — Minimum retrieval: Embeddings + vector store + keyword/BM25 + evaluation dataset.

Phase 2 — Metrics: Precision@K, Recall@K, MRR and NDCG@K before optimization.

Phase 3 — Hybrid + RRF: Combine sources and establish the baseline.

Phase 4 — Reranking and quality: Embedding reranker, quality scoring, duplicates and diversity.

Phase 5 — LTR: FeatureExtractor + model + ranker + feedback dataset.

Phase 6 — Persistence: ModelRepository, durable feedback and restart recovery.

Phase 7 — Learning: Offline TrainingService + OnlineTrainer/Optimizer.

Phase 8 — Context-aware: Query profile, strategy, candidate budget and ContextModel.

Phase 9 — Knowledge graph: Extraction, synchronization, inference and graph evidence.

Phase 10 — Lifecycle: Importance, freshness, consolidation and cleanup.

Phase 11 — Operations: Scheduler, jobs, retries, locking, metrics and dashboard.

Phase 12 — Release gate: Regression datasets, CI, container persistence, security and API contracts.

23. Acceptance criteria for a stable release

- Ranking meets or exceeds baseline thresholds for Precision@K, Recall@K, MRR and NDCG@K.
- A restart preserves memory, LTR model, feedback, knowledge graph and inference state.
- Feedback semantics distinguish impression, click, accept, reject and ignore.

- The pipeline records per-stage latency and final result quality.
- Regression tests are reproducible and never write into real production data.
- ContextModel is connected to the real path and absence of context preserves baseline behavior.
- Knowledge extraction synchronizes the graph idempotently.
- Jobs have persistent history, locking/retry and ordered dependencies.
- No active legacy path competes with the canonical runtime path.

24. Conclusion

This project should not be described as a simple AI application. It is an adaptive memory and retrieval system with multiple retrieval sources, learned ranking, feedback, persistence, context and structured knowledge.

Its most reusable design principle is the separation of responsibilities: retrieve, fuse, rank, rerank, score quality, reduce redundancy, learn from feedback, persist learning and verify improvement with regression metrics.

Because each phase has a clear contract and measurable output, embeddings, rankers, optimizers, storage engines or context strategies can evolve without replacing the entire system.